

TECHCATALYST DE 2026

Polars and DataFrame Alternatives

Keep the data rules, then choose an execution model that fits the workload.

Week 3 Polars

Express

Plan

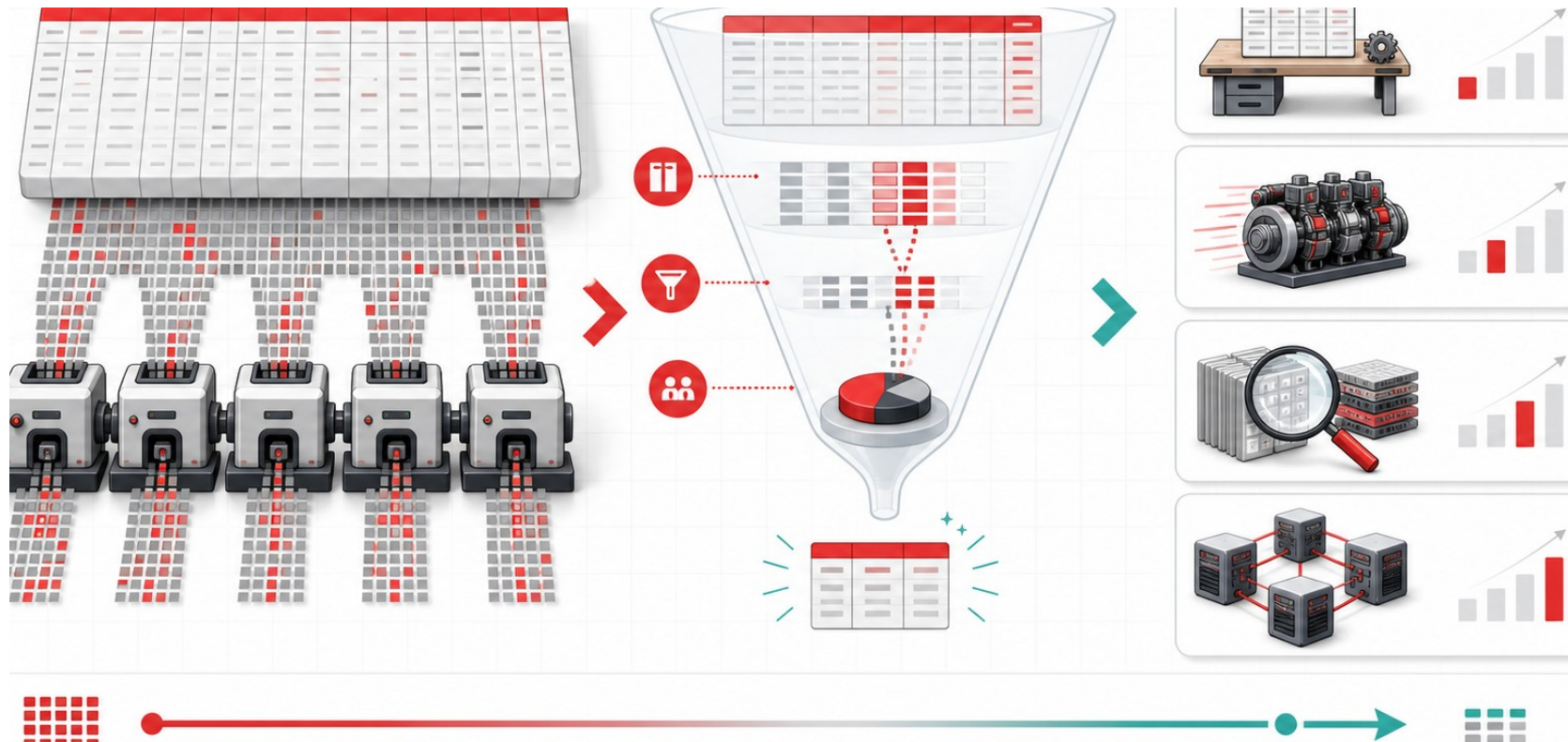
Optimize

Collect

Compare

Choose

Lazy execution reduces work before it begins



What you will be able to do

Learning goals	By the end, you can...
Transfer	Translate Pandas intent into Polars expressions
Select	Use select, filter, and with_columns explicitly
Aggregate	Use group_by and expression-based aggregation
Plan	Explain the difference between eager and lazy execution
Inspect	Use explain before collect to study the query plan
Optimize	Recognize projection and predicate pushdown
Compare	Distinguish Pandas, Polars, DuckDB, and Dask workloads
Choose	Start with the smallest engine that fits the evidence

Weeks 1 to 4 are an AI-free zone

You cannot review code you could not have written.

What is allowed

- Write and debug your own code
- Use official docs and course references
- Ask peers to explain, not complete
- Call APIs and SDKs when the lesson requires them

Why we do this

- Fluency comes from typing and testing
- Error messages build debugging judgment
- Interviews still test fundamentals
- Later AI use should amplify skills you already own

01

Polars and alternatives

Keep the data rules. Change the execution model.

Polars changes the expression model

What changes: column expressions replace index-centered selection and many Python callbacks.

What improves: lazy queries expose the full plan so the optimizer can push filters and projections toward the scan.

What stays: define the grain, enforce types, validate row counts, and make cleaning rules explicit.

Translate the intent, not each keystroke

Pandas

- `df.loc[mask, cols]`
- `df.groupby("station").agg(...)`
- Familiar, flexible, eager by default
- Strong choice for exploration and teaching

Polars

- `df.filter(...).select(...)`
- `df.group_by("station").agg(...)`
- Expression-first, eager or lazy
- Strong choice for columnar local pipelines

Lazy execution lets the engine optimize the plan

```
query = (  
    pl.scan_csv("data/raw/weather_raw.csv")  
    .filter(pl.col("date").is_not_null())  
    .select("station", "date", "tmax", "tmin", "prcp")  
    .group_by("station")  
    .agg(pl.col("prcp").sum().alias("total_prcp"))  
)  
  
print(query.explain())  
result = query.collect()
```

scan builds a query plan. collect executes it. explain shows whether Polars pushed filters and column selection toward the data source.

Choose the smallest engine that fits the workload

Tool	Execution	Best fit	Teaching stance
Pandas	Eager, in memory	Exploration, moderate data	Default foundation
Polars	Eager or lazy	Fast local pipelines	Learn the expression model
DuckDB	Lazy SQL	SQL over files and frames	Bridge into Week 4
Dask	Partitioned	Distributed or out- of-core	Use after Pandas no longer fits

Start simple. Change tools only when workload evidence demands it.

Select and filter with expressions

```
query = (  
    pl.scan_csv("data/raw/weather_raw.csv")  
    .filter(pl.col("date").is_not_null())  
    .select("station", "date", "tmax", "tmin", "prcp")  
)  
  
print(query.explain())
```

Expressions describe work on columns. `scan_csv` builds a lazy plan, so Polars can move filters and column selection closer to the source.

Add columns and aggregate as expressions

```
result = (  
    query.with_columns(  
        (pl.col("tmax") - pl.col("tmin")).alias("temp_range")  
    )  
    .group_by("station")  
    .agg(  
        pl.len().alias("days"),  
        pl.col("prcp").sum().alias("total_prcp"),  
    )  
    .collect()  
)
```

with_columns creates or replaces columns.
group_by uses expressions for aggregation.
collect executes the optimized plan.

Match the workload to the engine

Workload signal	Start with	Why
Exploration that fits memory	Pandas	Familiar and flexible
Columnar local pipeline	Polars	Expressions and lazy optimization
SQL over files or frames	DuckDB	Queries Parquet and DataFrames directly

Start with the smallest engine that fits the evidence. Move only when the workload changes.

Use Polars as a confidence transfer

Goal: reproduce the same 14-row silver result with a different execution model.

Run the optional path only after
the Pandas result is verified.

- Scan the raw CSV lazily
- Express the same date, duplicate, and missing-value rules
- Create temp_range and is_rainy with expressions
- Inspect the plan, collect, and write Parquet
- Compare row counts and schema with the Pandas output

Definition of done: the result matches the Pandas business rules, and your note explains one API difference plus one execution difference.

Translate one trusted pipeline

Goal: reproduce a familiar Pandas result with Polars and explain the execution difference.

25 to 35 min · pairs

- Translate select, filter, with_columns, and group_by
- Build the query with scan_csv
- Inspect the optimized plan with explain
- Collect and compare row counts with Pandas

Stretch: run the optional Polars silver path and prove it produces the same 14 rows.

Deliverable: matching output plus a two-sentence tool recommendation based on evidence.

Polars official references

Reference	Use it for
Polars: Coming from Pandas	Expressions, indexes, and API differences
Polars lazy API	Query plans, explain, collect, and pushdown
Polars expressions guide	select, filter, with_columns, and aggregation
DuckDB Python API	SQL over files and DataFrames
Dask DataFrame best practices	When distributed execution is justified

Checked July 10, 2026. Prefer official documentation when older examples or remembered syntax disagree.

Change engines without losing judgment

1

Translate intent, not individual Pandas keystrokes.

2

Polars expressions describe column work without an index-centered model.

3

Lazy plans can push filters and projections toward the scan.

4

Use explain before collect when optimization matters.

5

Choose Pandas, Polars, DuckDB, or Dask from workload evidence.

Exit ticket: Which workload signal would justify moving beyond Pandas?